L'EDITOR DELLA SUITE

bm Pixel

Sprite, tileset e hitbox: disegna la grafica di un gioco bm direttamente sulla console.

Indice

Cos'è bm Pixel **4**

Cosa fa 4

Perché conta 4

Da sapere 4

Attenzione 4

Aprire un file **5**

Da dove partire 5

Progetti e giochi 5

Il menu (Esc) 5

Le pagine **6**

F1: disegno 6

F2: lo sheet 6

F3: la palette 6

F4: lo stack 6

L'anteprima delle hitbox (j) 7

Gamepad 7

Strumenti di disegno e tasti **8**

Cosa fa 8

I tasti della pagina di disegno 8

Forme e selezioni 9

Copiare, spostare e trasformare 9

L'assistente (F6) 9

Sprite, zone, fotogrammi e fps **10**

Cosa fa 10

Come bm Pixel mostra i fotogrammi 10

Da sapere 10

Attenzione 10

Flag delle tile **11**

Cosa fa 11

La convenzione . . . 11
Da sapere . . . 11
Attenzione . . . 11

Hitbox e hurtbox . . . 12
Cosa fa . . . 12
Come li mostra bm Pixel . . . 12
Attenzione . . . 12

Palette . . . 13
Cosa fa . . . 13
I tasti di F3 . . . 13
Da sapere . . . 13
Attenzione . . . 13

Importare ed esportare risorse . . . 14
Cosa fa . . . 14
Cosa puoi fare oggi . . . 14
Attenzione . . . 14

Tutorial guidati . . . 15
Disegna il tuo primo sprite . . . 15
Crea uno sprite animato . . . 15
Segna le tile solide . . . 15
Aggiungi una hitbox . . . 16

Consigli e limiti . . . 17
Limiti . . . 17
Consigli . . . 17
Quando qualcosa va storto . . . 17

Punti aperti . . . 18

LEGGIMI

Cos'è. Il manuale di **bm Pixel**, l'editor di pixel art integrato nella console. Lavora sullo sprite sheet di un gioco o di un progetto.

Come usarlo. Leggi una volta i primi tre capitoli, poi segui i passi numerati dei tutorial. Ciò che sta in un riquadro con bordo e ombra è fatto per essere copiato così com'è.

Cosa manca. Dove bm Pixel non fa ancora qualcosa, o il repository non dice come, il manuale mostra un'etichetta evidenziata DA COMPLETARE. Non si inventa nulla.

Cos'è bm Pixel

bm Pixel è lo strumento di disegno della suite bm. Disegni sprite e tile sullo **sprite sheet**: l'unica immagine che un gioco legge con `spr`, `sspr` e `map`, e che bm Studio usa come texture dei suoi modelli.

Cosa fa

- Disegna uno sprite ingrandito, pixel per pixel, con matita, gomma, riempimento, linee, rettangoli, ovali, selezioni e specchio.
- Riproduce lo sprite e quelli accanto come animazione, con onion skin.
- Mostra tutto lo sheet, ti lascia scegliere uno sprite, copiarlo e ridimensionare lo sheet.
- Modifica la palette fino a 256 colori.
- Mostra sopra il disegno le hitbox e le hurtbox delle zone dello sheet.

Perché conta

Un gioco ha l'aspetto di ciò che c'è sul suo sheet. Con bm Pixel disegni sulla console stessa, provi il gioco con un tasto (F5) e torni nello stesso punto.

Da sapere

- Le pagine sono quattro: **F1** disegno, **F2** sheet, **F3** palette, **F4** stack. **Esc** apre il menu.
- Tieni premuto **F12**, o premi **?**, per vedere i tasti della pagina. **Tab** (sul pad **X**) apre l'elenco dei comandi della pagina.
- Con il mouse, il mouse fa ciò che fanno i tasti.

Attenzione

- bm Pixel modifica lo sheet e la palette. Crea solo zone stack (F4). Non dà nomi alle altre zone, non disegna hitbox e non imposta i flag delle tile (vedi i capitoli dedicati).

Aprire un file

Da dove partire

Parti dalla scheda **Dev** del menu, oppure premi **X** sulla copertina di un gioco e scegli **Open in bm Pixel**. Se avvii bm Pixel senza file, mostra l'elenco dei progetti `.bme` di `/bme` e dei file `.bm` e `.bme` che trova in `/carts` e nella radice della scheda, e dietro c'è già un nuovo sheet vuoto.

Progetti e giochi

- Un **progetto** è un file `.bme`. Gli editor cambiano i progetti e li salvano dove sono.
- Un **gioco** è un file `.bm` (o `.b16`). È di sola lettura. Quando ne apri uno, lo schermo dice `a game: saving makes its copy`, e salvando si scrive una copia `.BME` modificabile, mai il gioco stesso.
- Nel file salvato cambiano solo lo sheet e la palette: codice, mappa, copertina, suoni e modelli restano come erano.

Il menu (Esc)

Voce	Tasto	Cosa fa
Continue		torna alla pagina
Open...	Ctrl+O	apre un altro `.bm` o `.bme`
New sheet	Ctrl+N	un nuovo sheet vuoto di 256x256
Save	Ctrl+S	salva il file
Save as...	Ctrl+Shift+S	chiede un nome di file (8.3, in `/carts`) e salva lì
Try the game	F5	salva se serve, avvia il gioco e ti riporta alla stessa pagina
Sheet size...		come **Shift+R** nella pagina sheet
Back to bm SDK		torna a bm SDK; compare solo quando bm Pixel è stato aperto dall'SDK
Exit bm Pixel		esce; se ci sono modifiche non salvate, sceglila di nuovo per confermare

Ctrl+Z e Ctrl+Y annullano e ripetono. Per ogni file bm Pixel ricorda lo sprite, la sua misura, l'animazione, il colore e la pagina.

NOTA

Un nuovo sheet salvato con *Save as* diventa una cartuccia il cui codice si limita a mostrare lo sheet. Sostituisci quel codice con quello del tuo gioco.

Le pagine

F1: disegno

Lo sprite scelto, ingrandito a sinistra. A destra: i colori, lo strumento, il numero dello sprite con la sua posizione sullo sheet, lo sprite come lo disegna il gioco e, accanto, l'animazione. Una riga sotto indica la velocità e se pausa, onion skin, box e specchio sono attivi.

[DA COMPLETARE: shots/editor-pixel-draw.png]

La misura dello sprite è 8, 16, 32, 64 o 128 pixel: **z** passa alla misura successiva. **PgUp** e **PgDn** vanno allo sprite prima e dopo. **t** mostra o nasconde la griglia.

F2: lo sheet

Tutto lo sheet. Le frecce scelgono uno sprite (nella misura scelta), **Enter** lo disegna. **+** e **-** fanno zoom. I fotogrammi dell'animazione sono contornati dopo lo sprite.

[DA COMPLETARE: shots/editor-pixel-sheet.png]

F3: la palette

Tutti i colori, fino a 256, con i valori di rosso, verde e blu di quello scelto. Vedi il capitolo sulle palette.

F4: lo stack

Una *zona stack* è una zona con *n* fette della stessa misura, una accanto all'altra sullo sheet, da sinistra a destra. La fetta 1 è la più bassa. Nella cartuccia è la riga `stack nome x y w h n` ; un gioco la disegna con `zstack` . La pagina Stack mostra a sinistra una fetta, ingrandita. A destra mostra le zone stack, una fila di piccole immagini delle fette e lo stack intero che gira (`stack` con un angolo di rotazione). Lo stack legge lo sheet a ogni fotogramma: una modifica a una fetta si vede subito. Una zona `stackv` ha le fette una sotto l'altra sullo sheet, dall'alto in basso; la pagina legge e scrive tutte e due le disposizioni. La riga finisce con `h` (una accanto all'altra, `stack`) o `v` (una sotto l'altra, `stackv`).

Tasto	Azione
sinistra / destra	la fetta prima / dopo
su / giù	la zona stack prima / dopo
PgUp / PgDn	8 fette
Enter	disegna la fetta in F1 (solo fette quadrate; **F4** torna qui)
k	onion skin: la fetta prima, attenuata, nei pixel vuoti di questa fetta
c	copia questa fetta sulla successiva e ci va (estrusione; **Ctrl+Z** annulla)
Del o x	cancella la zona (premi due volte; i pixel restano sullo sheet)
n	nuova zona: nome, x, y, w, h, n, `h` o `v` (per esempio `tree 0 0 16 16 8 h`)

Tasto	Azione
e	modifica la zona (la stessa riga; un nome nuovo crea una zona nuova)
p	lo stack gira o si ferma
, e .	gira lo stack a mano

Se non esiste nessuna zona stack, **n** crea la prima. La riga parte dallo sprite scelto in F1. **Ctrl+S** scrive le zone e il numero di fette (byte 26 della zona) insieme allo sheet. La riga della zona non ha l'annulla; la copia di una fetta sì. L'onion skin disegna al massimo 600 quadratini, solo quando è attivo. Con il mouse: clic su una zona o su una piccola immagine per sceglierla, doppio clic su una piccola immagine per disegnarla, la rotella cambia fetta, trascinare sullo stack che gira lo ruota a mano, un clic sopra lo ferma o lo fa ripartire, il tasto destro apre l'elenco dei comandi. Sul pad: sinistra / destra la fetta, su / giù la zona, **A** disegna la fetta, **B** gira o ferma, **X** + sinistra / destra gira a mano, **X** (tocco) l'elenco dei comandi.

L'anteprima delle hitbox (j)

Premi **j** in F1 o F2 per mostrare o nascondere i box della sezione `BOXES` del file. Il messaggio dice `hitboxes on (BOXES)` oppure `hitboxes off`. Vedi il capitolo sulle hitbox.

Gamepad

Tasti	Azione
Y + sinistra / destra	cambia pagina (F4 è una pagina)
Y + B	menu
Y + A	annulla
Y + su / giù	lo sprite prima / dopo
D-pad	il puntatore
A	disegna (tenuto: un tratto)
B	colore dallo sprite, oppure annulla
X + sinistra / destra	il colore
X + su / giù	lo strumento
X (tocco)	l'elenco dei comandi

Strumenti di disegno e tasti

Cosa fa

In F1 un puntatore si muove con le frecce. **Spazio** usa lo strumento. Con matita o gomma, tenendo premuto spazio si disegna un tratto.

I tasti della pagina di disegno

Tasto	Azione
frecce	muovono il puntatore
spazio	usa lo strumento
b	matita
e	gomma
g	riempimento
i	prende il colore di un pixel (un colore nuovo va nella palette)
l	linea
u / U	rettangolo / rettangolo pieno
o / O	ovale / ovale pieno
m	seleziona
, e .	il colore prima / dopo
x	il colore usato prima
1 - 9, 0	i primi dieci colori
y	disegno speculare, sinistra-destra
t	griglia
k	onion skin
p	avvia o ferma l'animazione
+ / -	più / meno fotogrammi
> / <	più veloce / più lento
z	misura dello sprite
PgUp / PgDn	sprite prima / dopo
j	mostra o nasconde i box
F6	l'assistente: una base per uno sprite
Tab	l'elenco dei comandi

Il colore 0 è trasparente.

Forme e selezioni

Per linee, rettangoli, ovali e selezioni: premi **spazio** per fissare un angolo, porta il puntatore all'altro angolo, premi di nuovo **spazio**. **Esc** annulla.

Copiare, spostare e trasformare

Tasto	Azione
Ctrl+C / Ctrl+X / Ctrl+V	copia, taglia, incolla (il blocco fluttua: muovilo con le frecce, posalo con spazio o Enter)
Enter	solleva la selezione per spostarla
h / v	specchia sinistra-destra / alto-basso
r	ruota di un quarto
Del	cancella la selezione, o tutto lo sprite
Shift+w a s d	fa scorrere lo sprite di un pixel (ciò che esce rientra dall'altro lato)

I pixel trasparenti di un blocco incollato lasciano ciò che c'è sotto.

L'assistente (F6)

F6 chiede una parola ("slime", "moneta", "astronave") e disegna una base con i colori della tua palette. Fluttua come un incolla: muovilo, poi premi Enter. Se non è disponibile, lo schermo dice `the assistant is not here`.

Sprite, zone, fotogrammi e fps

Cosa fa

Una **zona** è una parte dello sheet con un nome: una posizione, una misura, un numero di **fotogrammi** (da 1 a 16) e una velocità in **fps** (0 è fermo). Un gioco disegna una zona con `zspr("hero_run", x, y)`. I fotogrammi sono riquadri della stessa misura, affiancati sulla stessa riga, a partire dal primo.

Come bm Pixel mostra i fotogrammi

- Lo sprite scelto e quelli che lo seguono nello sheet sono i fotogrammi dell'animazione. **+** e **-** ne impostano il numero (da 1 a 16), **>** e **<** la velocità (da 1 a 30 fps), **p** mette in pausa.
- Il riquadro a destra li riproduce, accanto allo sprite a grandezza reale.
- Quando lo sprite su cui sei coincide con una zona del file (primo passaggio: la stessa riga, la misura dello sprite uguale a quella del fotogramma della zona e la colonna in un punto qualsiasi dell'estensione dei fotogrammi della zona, quindi vale uno qualsiasi dei suoi fotogrammi; altrimenti, secondo passaggio: lo stesso angolo con qualsiasi misura, che mostra il fotogramma 1), il pannello mostra `zone "name"` `frame n/m` al posto della chiamata `spr(...)`.

Da sapere

- I fotogrammi e la velocità che imposti sono ricordati da bm Pixel per ogni file, ma non vengono scritti nella cartuccia.
- Perché i fotogrammi corrispondano a una zona, disegnali uno dopo l'altro da sinistra a destra.

Attenzione

- [DA COMPLETARE: come dare un nome a una zona e assegnarle fotogrammi e fps. Le vecchie note dicono che bm Pixel crea le zone, ma il codice di F1, F2 e F3 non ha un tasto per farlo; oggi le zone si scrivono in un file di testo (`name x y width height [frames [fps]]`) e si passano a `mkbm.py --sprites`]

DA COPIARE

```
# zones.txt: nome x y larghezza altezza [fotogrammi [fps]]
hero_still   0  32 16 16
hero_run     16 32 16 16 4 10
coin         0  48 8  8  6 12
```

Flag delle tile

Cosa fa

Ogni tile dello sheet (ogni cella 8x8) ha 8 flag, numerati da 0 a 7. Un gioco guarda i flag, non i numeri delle tile, per sapere cos'è una tile.

La convenzione

Flag	Valore	Significato
0	1	solida
1	2	piattaforma: si attraversa dal basso
2	4	scala
3	8	acqua
4	16	fa male
5 - 7		liberi

È la convenzione di bmlib; ogni gioco può sceglierne un'altra. Il gioco legge un flag con `fget(n, f)` e i flag sotto un rettangolo con `mflags`.

Da sapere

- I flag sono salvati nella sezione `FLAGS` del file, in base alla posizione della cella: uno sheet ingrandito in seguito li conserva.
- Tile su una griglia 8x8: disegna le tile a misura 8 (**z**) così uno sprite è una tile.

Attenzione

- bm Pixel non mostra né imposta i flag. **[DA COMPLETARE: i tasti esatti per impostare i flag di una tile nella pagina sprite di bm SDK (le note dicono che i tasti da 0 a 7 accendono i flag della cella) e come li mostra lo schermo]**

Hitbox e hurtbox

Cosa fa

La sezione `BOXES` contiene, per ogni fotogramma di una zona, i rettangoli che un gioco usa in un combattimento: dove il personaggio può essere colpito, dove il suo colpo fa male, lo spazio che occupa.

Tipo	Numero	Colore sullo schermo
hurt: dove può essere colpito	0	blu
hit: dove il colpo fa male	1	rosso
body: lo spazio che occupa	2	verde
altro: del gioco	da 3 a 255	giallo

Un box è dato dall'angolo in alto a sinistra del fotogramma, in pixel, e può uscirne. Un box con fotogramma 0 vale per ogni fotogramma della zona.

Come li mostra bm Pixel

- **j** attiva e disattiva i box (all'inizio sono attivi).
- In F1 sono disegnati sopra lo sprite ingrandito e sopra le due piccole anteprime (lo sprite, e il fotogramma su cui è l'animazione).
- In F2 sono disegnati sopra ogni fotogramma di ogni zona visibile.
- Un box compare solo quando lo sprite su cui sei coincide con una zona del file.

[DA COMPLETARE: shots/editor-pixel-boxes.png]

Attenzione

- In bm Pixel non puoi disegnare, spostare o ridimensionare un box. [DA COMPLETARE: come creare i box sulla console. Oggi si scrivono sotto la zona nel file di testo delle zone, con righe `hurt`, `hit` o `body`, il fotogramma (`*` per tutti) e il rettangolo x, y, larghezza, altezza]

DA COPIARE

```
punch        0  64 32 32 4 12
  hurt  *    8  2 16 30         # il corpo, in ogni fotogramma
  hit   3    24 10 10  6        # il pugno esce nel fotogramma 3
```

Palette

Cosa fa

La palette è l'elenco dei colori con cui disegni, fino a 256. Viene salvata con lo sheet: quando lo sheet ha al massimo 256 colori diventa una sezione `SHEET8` con la tua palette per prima, e riaprendo il file torna la stessa palette. La leggono anche i giochi, l'SDK, bm Studio e bm Animator.

I tasti di F3

Tasto	Azione
frecce	scelgono un colore
Enter	disegna con quello (torna a F1)
e	lo modifica: su / giù scelgono R, G o B; sinistra / destra cambiano di 8; `<` e `>` di 1; Enter o Esc per finire
a	aggiunge un colore dopo quello scelto
Del	lo rimuove (ne resta almeno uno)
[e]	lo spostano
s	ordina per tinta
f	prende i colori usati nello sheet (i 256 più usati)
1	la palette dell'SDK
2	la palette di bm Studio
x	cambia il colore di disegno in quello scelto, in tutto lo sprite
X	lo stesso, in tutto lo sheet

Da sapere

- La console tiene 16 bit per pixel (RGB565). Accanto a ogni colore lo schermo mostra come appare sulla console.
- Un pixel che non ridisegni mantiene il colore che aveva nel file; un pixel che ridisegni prende il colore della palette.

Attenzione

- **X** non funziona su uno sheet più grande di 512x512: usa **x** nello sprite.

Importare ed esportare risorse

Cosa fa

Una risorsa è una cosa estratta da una cartuccia in un file suo. Per le immagini il file è un `.bmi` : lo sheet (o un suo pezzo), le zone di `SPRITES` , i loro `BOXES` e i `FLAGS` delle tile. Una palette è un `.bmc` .

File	Contiene
`.bmi`	immagini e sprite: sheet, SPRITES, BOXES, FLAGS
`.bmc`	una palette
`.bmk`	un kit: più tipi insieme su un unico sheet condiviso

Cosa puoi fare oggi

- Nella scheda **Lib** del menu, il gruppo **Images** elenca le zone di SPRITES. `docs/RISORSE.md` descrive **A** che ne apre una in bm Pixel, ma nel codice non è stato confermato. La scheda è nascosta a meno che `lib_tab=1` sia in `bm/config.txt` .
- Su PC, `scripts/bmres.py` estrae una zona di sprite in un `.bmi` (con i suoi box), aggiunge una risorsa a un `.bm` e converte da e verso `.png` , `.hex` e `.gpl` .

DA COPIARE

```
python3 scripts/bmres.py list GAME.bm
python3 scripts/bmres.py extract GAME.bm --sprite coin -o COIN.bmi
python3 scripts/bmres.py add GAME.bm COIN.bmi
```

Attenzione

- bm Pixel non ha un comando di importazione o esportazione. [DA COMPLETARE: se bm Pixel apre e salva direttamente file `.bmi` e `.bmc` ; le note lo prevedono]
- Aggiungere una risorsa a un `.bm` non toglie mai nulla, e a un nome già esistente viene aggiunto un numero.

Tutorial guidati

Disegna il tuo primo sprite

1. Avvia bm Pixel dalla scheda **Dev**. Se si apre l'elenco dei file, premi **Esc** per chiuderlo; il nuovo sheet vuoto c'è già. (Più avanti, **Ctrl+N** ne crea un altro.)
2. Premi **F1**. Hai un nuovo sheet di 256x256 e la palette dell'SDK.
3. Premi **z** finché il pannello dice `16x16`. Premi **t** se vuoi la griglia.
4. Premi **.** più volte finché un colore ti piace, oppure premi da **1** a **9, 0**. Il pannello mostra `colour n`.
5. Premi **b** per la matita. Muovi il puntatore con le frecce e premi **spazio**. Tieni premuto spazio e muoviti per disegnare una linea di pixel.
6. Premi **y** per disegnare in modo speculare: ciò che disegni da un lato appare dall'altro. Premi di nuovo **y** per fermarlo.
7. Se sbagli, premi **Ctrl+Z**.
8. Premi **Esc**, scegli **Save as...**, scrivi un nome come `hero.bme` e premi Enter. Lo schermo dice `saved`.

[DA COMPLETARE: shots/editor-pixel-first.png]

> **PROVA**
> Disegna una moneta: un ovale pieno con **O**, poi scegli un colore più scuro e disegna il contorno con **o**.

Crea uno sprite animato

1. Apri il tuo sprite in F1 a misura 16x16. È il primo fotogramma.
2. Premi **PgDn**: sei sullo sprite successivo, a destra sullo sheet. Disegna il secondo fotogramma.
3. Premi di nuovo **PgDn** per un terzo fotogramma, e così via. I fotogrammi devono stare uno accanto all'altro, da sinistra a destra.
4. Premi **PgUp** finché sei tornato al primo fotogramma.
5. Premi **+** finché il pannello indica il numero di fotogrammi che hai disegnato. Il riquadro a destra li riproduce.
6. Premi **>** o **<** finché la velocità ti sembra giusta. La riga sotto il pannello indica gli fps. **p** mette in pausa.
7. Premi **k** per l'onion skin: il fotogramma precedente si vede sotto quello che stai disegnando.
8. Salva con **Ctrl+S**.

> **ATTENZIONE**
> Il numero di fotogrammi e gli fps che imposti sono ricordati da bm Pixel per questo file, ma non vengono scritti nella cartuccia. Per usarli in un gioco come sprite con nome, vedi il capitolo sulle zone.

Segna le tile solide

1. In bm Pixel, disegna la tua tile a misura 8 (**z**): un muro, un pavimento. Salva con **Ctrl+S**.
2. Apri il progetto in bm SDK (il file conserva il suo sheet).

3. Nella pagina sprite, scegli la cella della tile del muro e accendi il flag 0 (i tasti da **0** a **7** servono per i flag).
4. Nel tuo codice, leggilo: `fget(mget(cx, cy), 0)` è `true` per una cella solida.

I flag sono nella tabella del capitolo sui flag delle tile: 0 solida, 1 piattaforma, 2 scala, 3 acqua, 4 fa male.

> **NOTA**
> Il passo 3 è quello che le note descrivono in una sola riga. Controlla sulla tua console come l'SDK mostra i flag, e vedi la lacuna nel capitolo sui flag delle tile.

Aggiungi una hitbox

1. Scrivi una zona nel tuo `zones.txt` e, sotto, i suoi box (vedi il capitolo sulle hitbox).
2. Costruisci la cartuccia con `mkbm.py --sprites zones.txt` e copiala sulla scheda.
3. Aprila in bm Pixel e vai in F1. Premi **z** finché la misura è quella del fotogramma della zona (16 per una zona 16x16).
4. Vai al primo fotogramma della zona con le frecce in F2, oppure con **PgUp** e **PgDn** in F1. Il pannello mostra `zone "name" frame 1/n`.
5. Controlla che **j** dica `hitboxes on (BOXES)`. I rettangoli compaiono sopra lo sprite: blu per hurt, rosso per hit, verde per body.
6. Premi **PgDn** per vedere i box del fotogramma successivo. Se un box è nel posto sbagliato, cambia i numeri in `zones.txt` e ricostruisci.

> **PROVA**
> Apri la pagina sheet (F2), premi **j** e guarda tutti i box di tutte le zone insieme.

Consigli e limiti

Limiti

Cosa	Limite
Misure degli sprite	8, 16, 32, 64, 128 pixel
Misura dello sheet	multipli di 8, da 8 fino a 4096
Nuovo sheet	256x256
Palette	da 1 a 256 colori
Fotogrammi dell'animazione	da 1 a 16, velocità da 1 a 30 fps
Zone in un file	da 1 a 1024
Box in un file	da 1 a 4096
File ricordati	gli ultimi 6

Consigli

- Tieni i fotogrammi di un'animazione uno accanto all'altro sulla stessa riga.
- Disegna le tile a misura 8, così uno sprite è una tile.
- Per cambiare la misura dello sheet: **Shift+R** in F2. Ciò che ci sta resta; il resto viene tagliato.
- Uno sheet grande impiega qualche secondo a salvarsi; lo schermo dice `saving ...` .
- Usa **x** e **X** in F3 per cambiare un colore in un altro senza ridisegnare.
- Prova spesso il gioco con **F5**: torni nella stessa pagina.

Quando qualcosa va storto

- `cannot open ...` : il file non è un `.bm` o `.bme` valido. Aprine un altro.
- `save failed: ...` : il messaggio dice perché; prova un altro nome con **Save as...**.
- `unsaved changes: choose again ...` : scegli di nuovo la stessa voce per confermare, oppure salva prima.

Punti aperti

- **Le pagine**: immagine da fare: shots/editor-pixel-draw.png
- **Le pagine**: immagine da fare: shots/editor-pixel-sheet.png
- **Sprite, zone, fotogrammi e fps**: come dare un nome a una zona e assegnarle fotogrammi e fps. Le vecchie note dicono che bm Pixel crea le zone, ma il codice di F1, F2 e F3 non ha un tasto per farlo; oggi le zone si scrivono in un file di testo (`name x y width height [frames [fps
- **Flag delle tile**: i tasti esatti per impostare i flag di una tile nella pagina sprite di bm SDK (le note dicono che i tasti da 0 a 7 accendono i flag della cella) e come li mostra lo schermo
- **Hitbox e hurtbox**: immagine da fare: shots/editor-pixel-boxes.png
- **Hitbox e hurtbox**: come creare i box sulla console. Oggi si scrivono sotto la zona nel file di testo delle zone, con righe `hurt`, `hit` o `body`, il fotogramma (`*` per tutti) e il rettangolo x, y, larghezza, altezza
- **Importare ed esportare risorse**: se bm Pixel apre e salva direttamente file `.bmi` e `.bmc`; le note lo prevedono
- **Tutorial guidati**: immagine da fare: shots/editor-pixel-first.png